In [3]:
```
#ran on hipergator tensorflow 2.18

import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow import keras
import tensorflow_datasets
import tensorflow_io
import librosa
```

In [4]:
```
def train(data="training_data_eee4773.npy", labels="training_labels_eee4773.npy", model_


    data = np.load(data).T
    t = np.load(labels)



    X_16k = np.array([librosa.resample(x, orig_sr=48000, target_sr=16000) for x in data]

    training_speech, validation_speech = tensorflow_datasets.load("speech_commands", spl


    #preprocessing data from the google speech commands
    #it is saved as a tensorflow dataset object so you can preprocess and batch it befor
    def preprocess(audio, label):
        audio = tf.cast(audio, tf.float32) / 32768.0
        audio = tf.reshape(audio, [-1])

        current_length = tf.shape(audio)[0]
        pad_total = tf.maximum(48000 - current_length, 0)
        pad_left = tf.random.uniform([], 0, pad_total + 1, dtype=tf.int32)
        pad_right = pad_total - pad_left

        audio = tf.pad(audio, [[pad_left, pad_right]])
        audio = audio[:48000]

        return audio, label


    training_speech = (
        training_speech
        .map(preprocess)
        .shuffle(1000)
        .batch(64)
    )

    validation_speech = (
        validation_speech
        .map(preprocess)
        .batch(64)
    )


    #creating encoder
    conv_encoder = keras.models.Sequential([
        keras.Input(shape=(48000,)),
```

```
    keras.layers.MelSpectrogram(fft_length=512,
                                sequence_stride=160,
                                sampling_rate=16000,
                                num_mel_bins=80,
                                min_freq=80,
                                power_to_db=True),

    keras.layers.Reshape((80, 301, 1)),

    keras.layers.Conv2D(filters=32,kernel_size=3, padding="same", activation="relu")
    keras.layers.MaxPool2D(pool_size=2, padding="same"),

    keras.layers.Conv2D(filters=64,kernel_size=3, padding="same", activation="relu")
    keras.layers.MaxPool2D(pool_size=2, padding="same"),

    keras.layers.Conv2D(filters=128,kernel_size=3, padding="same", activation="relu"
    keras.layers.MaxPool2D(pool_size=2, padding="same"),
])

#training encoder on the google speech dataset
inp = keras.layers.Input(shape=(48000,))

x = conv_encoder(inp)
x = keras.layers.Flatten()(x)
x = keras.layers.Dropout(0.3)(x)
x = keras.layers.Dense(500, activation='relu')(x)
x = keras.layers.Dropout(0.3)(x)
out = keras.layers.Dense(12, activation="softmax")(x)
#there is 12 classes in this



encoder_classifier = keras.models.Model(inp, out)


encoder_classifier.compile(loss='sparse_categorical_crossentropy',
              optimizer='adam', metrics=["accuracy"])

stopping = keras.callbacks.EarlyStopping(monitor="val_loss", patience=4, restore_bes

history = encoder_classifier.fit(training_speech, validation_data=validation_speech,


#training attached classifier
for layer in conv_encoder.layers:
        layer.trainable = True
inp = keras.layers.Input(shape=(48000,))

x = conv_encoder(inp)
x = keras.layers.Flatten()(x)

x = keras.layers.Dropout(0.5)(x)

x = keras.layers.Dense(32, activation='relu', kernel_regularizer=keras.regularizers.

x = keras.layers.Dropout(0.5)(x)
out = keras.layers.Dense(1, activation='sigmoid')(x)

clf = keras.models.Model(inp, out)
```

```
    #compiling, running, and returning encoder + attached classifier trained on wake wor
    clf.compile(loss='binary_crossentropy',
              optimizer="adam",
              metrics=["accuracy"])

    stopping = keras.callbacks.EarlyStopping(monitor="val_loss", patience=4, restore_bes



    #to have balanced validation
    X_train_16k, X_val_16k, t_train, t_val = train_test_split(X_16k, t, test_size=0.15,

    history = clf.fit(X_train_16k, t_train, epochs=30, batch_size=32, validation_data=(X


    clf.save(model_path)
    return clf
```

In [5]:
```
model = train(data = "training_data_eee4773.npy", labels= "training_labels_eee4773.npy",
```

```
Epoch 1/10
2026-04-22 12:03:19.504940: E external/local_xla/xla/stream_executor/cuda/cuda_driver.cc:
152] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: CUDA_ERROR_NO_DE
VICE: no CUDA-capable device is detected
2026-04-22 12:03:20.667331: I tensorflow/core/kernels/data/tf_record_dataset_op.cc:376] T
he default buffer size is 262144, which is overridden by the user specified `buffer_size`
of 8388608
```

```
1337/1337 ━━━━━━━━━━━━━━━━━━━━ 223s 166ms/step - accuracy: 0.6383 - loss: 3.5744 - val_ac
curacy: 0.8149 - val_loss: 0.5592
Epoch 2/10
1337/1337 ━━━━━━━━━━━━━━━━━━━━ 222s 166ms/step - accuracy: 0.7974 - loss: 0.6127 - val_ac
curacy: 0.8940 - val_loss: 0.3322
Epoch 3/10
1337/1337 ━━━━━━━━━━━━━━━━━━━━ 221s 166ms/step - accuracy: 0.8546 - loss: 0.4455 - val_ac
curacy: 0.9090 - val_loss: 0.2780
Epoch 4/10
1337/1337 ━━━━━━━━━━━━━━━━━━━━ 222s 166ms/step - accuracy: 0.8780 - loss: 0.3764 - val_ac
curacy: 0.9236 - val_loss: 0.2447
Epoch 5/10
1337/1337 ━━━━━━━━━━━━━━━━━━━━ 223s 167ms/step - accuracy: 0.8943 - loss: 0.3323 - val_ac
curacy: 0.9233 - val_loss: 0.2468
Epoch 6/10
1337/1337 ━━━━━━━━━━━━━━━━━━━━ 222s 166ms/step - accuracy: 0.9057 - loss: 0.2973 - val_ac
curacy: 0.9351 - val_loss: 0.2087
Epoch 7/10
1337/1337 ━━━━━━━━━━━━━━━━━━━━ 225s 168ms/step - accuracy: 0.9096 - loss: 0.2861 - val_ac
curacy: 0.9367 - val_loss: 0.2031
Epoch 8/10
1337/1337 ━━━━━━━━━━━━━━━━━━━━ 229s 171ms/step - accuracy: 0.9152 - loss: 0.2675 - val_ac
curacy: 0.9353 - val_loss: 0.2031
Epoch 9/10
1337/1337 ━━━━━━━━━━━━━━━━━━━━ 227s 170ms/step - accuracy: 0.9186 - loss: 0.2582 - val_ac
curacy: 0.9396 - val_loss: 0.1879
Epoch 10/10
1337/1337 ━━━━━━━━━━━━━━━━━━━━ 224s 168ms/step - accuracy: 0.9178 - loss: 0.2639 - val_ac
curacy: 0.9419 - val_loss: 0.1831
Epoch 1/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 2s 98ms/step - accuracy: 0.5297 - loss: 0.7470 - val_accuracy:
0.8519 - val_loss: 0.5151
Epoch 2/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 74ms/step - accuracy: 0.8594 - loss: 0.4011 - val_accuracy:
0.8148 - val_loss: 0.4834
Epoch 3/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 75ms/step - accuracy: 0.9040 - loss: 0.2959 - val_accuracy:
0.8704 - val_loss: 0.3907
Epoch 4/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 72ms/step - accuracy: 0.9334 - loss: 0.2230 - val_accuracy:
0.8519 - val_loss: 0.3358
Epoch 5/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 74ms/step - accuracy: 0.9738 - loss: 0.1023 - val_accuracy:
0.8704 - val_loss: 0.3520
Epoch 6/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 73ms/step - accuracy: 0.9671 - loss: 0.0810 - val_accuracy:
0.8889 - val_loss: 0.3776
Epoch 7/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 71ms/step - accuracy: 0.9786 - loss: 0.0656 - val_accuracy:
0.8889 - val_loss: 0.3851
Epoch 8/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 70ms/step - accuracy: 0.9880 - loss: 0.0543 - val_accuracy:
0.8704 - val_loss: 0.4250
```